

```
</ul>
</section>
</main>
```

Script.js is present in the public folder, as shown in the code listings below. It runs every time the HTML page is refreshed, and makes a server side query by invoking a GET request to the URL */counts* from the server. It then processes the response of the *counts* list sent by the server and populates the ** in the client HTML page. The resultant Web page will look like what's shown in Figure 3 – a dump of coronavirus statistics.

Fetch */counts* from our Web server and use an iterator to pass each count into our *appendNewCount()* method:

```
// fetch the initial list of dreams
fetch("/counts")
  .then(response => response.json()) //
  parse the JSON from the server
  .then(counts => {
    // remove the loading text
    countsList.firstChild.
    remove();

    // iterate through every dream and
    add it to our page
    counts.forEach(appendNewCount);

  });
```

countsList references our ** counts in our *index.html*. Our *appendNewCount()* method creates a new list item, sets *innerText* to *counts passed*, and appends the same into our *countsList* reference:

```
// client-side js, loaded by index.html
// run by the browser each time the
page is loaded
// define variables that reference
elements on our page
const countsList = document.
getElementById("counts");
// a helper function that creates a
list item for a given dream
function appendNewCount(count) {
```

```
const newListItem = document.
createElement("li");
newListItem.innerText = count;
countsList.appendChild(newListItem);
}
```

The resultant HTML gets displayed, as shown in Figure 3. This basically completes the client part of a coronavirus data scraper using Glitch. What we will see in subsequent sections is how to use the *dev* tools available in Chrome or browsers based on the Chrome OS like Edge, to build PWA auditing and make our Web page PWA compliant.

A Dump of latest Coronavirus stats

The latest stats

- cases 157,241
- deaths 5,844

Fetches from [Worldometer](#)

Figure 3: A dump of coronavirus statistics

Fulfilling the progressive Web app mandate

In the following sections, we look at adding more files and content to our project to make it PWA compliant. We will be able to certify our app as compliant by the Lighthouse audit tool. The Web app manifest does not meet the installation requirements, which basically states that to be a PWA and be able to tick off the advantages mentioned earlier, we need to register a manifest file for our Web app. In the following code segments, we see the *manifest.json* file added under the public folder with some explanations.

Chrome uses this to make an automatic splash screen, along with *background_color*, apart from also making the PWA SEO-friendly and searchable.

```
{
  "short_name": "COVID-19 stats",
```

```
"name": "novelCoronaVirus COVID-19:
latest stats on infection and death",
  "description": "novelCoronavirus
COVID-19 information"
```

Chrome needs a minimum of two icon files of sizes 192×192 and 512×512, which are used for showing the PWA and are also shown in the splash screen and as the installed icon.

```
"icons": [
  {
    "src": "https://cdn.
glitch.com/a812b85a-f21e-
4582-a22f-7d2a24581d91%2F192.
png?v=1583926151789",
    "type": "image/png",
    "sizes": "192x193"
  },
  {
    "src": "https://cdn.
glitch.com/a812b85a-f21e-
4582-a22f-7d2a24581d91%2F512.
png?v=1583924262338",
    "type": "image/png",
    "sizes": "512x512"
  }
],
```

display can be *standalone/ fullscreen / minimal-ui*. *scope* defines from what page the PWA manifest will be active. It should cover *start_url* also. *theme_color* sets the colour of the toolbar and should match the same value set in the *meta* of the *index.html* page.

```
"start_url": "/?source=pwa",
"background_color": "#3367D6",
"display": "standalone",
"scope": "/",
"theme_color": "#3367D6"
}
```

Also, our *index.html* needs to add the references given below to link to the *manifest.json* and add the *theme-color*, as shown below:

```
<link rel="manifest" href="/manifest.
json" />
```